Missão Prática | Nível 3 | Mundo 3

Marcelo Picado Schulze | 202403781506

Campus BANCÁRIOS
Nível 3: Back-end Sem Banco Não Tem – 202403781506 – 3° periodo - semestre 2025.1

Objetivo da Prática

Implementar persistência com base no middleware JDBC.

1.Utilizar o padrão DAO (Data Access Object) no manuseio de dados.

2.Implementar o mapeamento objeto-relacional em sistemas Java.

3.Criar sistemas cadastrais com persistência em banco relacional.

4.No final do exercício, o aluno terá criado um aplicativo cadastral com uso do SQL Server na persistência de dados.

1º Procedimento | Mapeamento Objeto-Relacional e DAO

```
CREATE TABLE Pessoa (
  idPessoa INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
  nome VARCHAR(255) NOT NULL,
  logradouro VARCHAR(255) NOT NULL,
  cidade VARCHAR(255) NOT NULL,
  estado VARCHAR(255) NOT NULL,
  telefone VARCHAR(20) NOT NULL,
  email VARCHAR(255) NOT NULL
);
GO

CREATE TABLE PessoaFisica (
  idPessoa INT NOT NULL PRIMARY KEY,
  cpf VARCHAR(14) NOT NULL UNIQUE,
  FOREIGN KEY (idPessoa) REFERENCES Pessoa(idPessoa)
);
GO
```

Estácio

```
CREATE TABLE PessoaJuridica (
  idPessoa INT NOT NULL PRIMARY KEY,
  cnpj VARCHAR(14) NOT NULL UNIQUE,
  FOREIGN KEY (idPessoa) REFERENCES Pessoa(idPessoa)
);
GO

CREATE TABLE Usuario (
  idUsuario INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
  email VARCHAR(255) NOT NULL,
  senha VARCHAR(255) NOT NULL
);
GO

CREATE TABLE Produto (
  idProduto INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
  nome VARCHAR(255) NOT NULL,
  quantidade INT NOT NULL,
  preco DECIMAL(10, 2) NOT NULL
);
GO

CREATE TABLE Movimentacao (
  idMovimentacao INT IDENTITY(1,1) PRIMARY KEY,
  idUsuario INT NOT NULL,
  idPessoa INT NOT NULL,
  idProduto INT NOT NULL,
  quantidade INT NOT NULL CHECK (quantidade > 0),
  tipo CHAR(1) NOT NULL CHECK (tipo IN ('E', 'S')),
  valorUnitario DECIMAL(10,2) NOT NULL CHECK (valorUnitario >= 0),
  dataMovimentacao DATETIME DEFAULT GETDATE(),

  FOREIGN KEY (idUsuario) REFERENCES Usuario(idUsuario),
  FOREIGN KEY (idPessoa) REFERENCES Pessoa(idPessoa),
  FOREIGN KEY (idProduto) REFERENCES Produto(idProduto)
```



```
);
GO
```

```
package cadastrobd.model;

public class Pessoa
{
    private int id;
    private String nome;
    private String logradouro;
    private String cidade;
    private String estado;
    private String telefone;
    private String email;

    public Pessoa() {}

    public Pessoa(String nome, String logradouro, String cidade, String estado, String
telefone, String email)
    {
        this.nome = nome;
        this.logradouro = logradouro;
        this.cidade = cidade;
        this.estado = estado;
        this.telefone = telefone;
        this.email = email;
    }

    public Pessoa(int id, String nome, String logradouro, String cidade, String estado,
String telefone, String email)
    {
        this.id = id;
        this.nome = nome;
        this.logradouro = logradouro;
        this.cidade = cidade;
        this.estado = estado;
        this.telefone = telefone;
        this.email = email;
    }

    public void exibir()
    {
        System.out.println("ID: " + id);
        System.out.println("Nome: " + nome);
        System.out.println("Logradouro: " + logradouro);
        System.out.println("Cidade: " + cidade);
        System.out.println("Estado: " + estado);
        System.out.println("Telefone: " + telefone);
        System.out.println("Email: " + email);
    }
```

```
    public int getId()
    {
        return id;
    }

    public void setID(int newID)
    {
        id = newID;
    }

    public String getNome()
    {
        return nome;
    }

    public String getLogradouro()
    {
        return logradouro;
    }

    public String getCidade()
    {
        return cidade;
    }

    public String getEstado()
    {
        return estado;
    }

    public String getTelefone()
    {
        return telefone;
    }

    public String getEmail()
    {
        return email;
    }
}
```

```
package cadastrobd.model;

public class PessoaFisica extends Pessoa
{
    private String cpf;

    public PessoaFisica()
    {
        super();
    }
```



```
    public PessoaFisica(String nome, String logradouro, String cidade, String estado,
String telefone, String email, String cpf)
    {
        super(nome, logradouro, cidade, estado, telefone, email);
        this.cpf = cpf;
    }

    public PessoaFisica(int id, String nome, String logradouro, String cidade, String
estado, String telefone, String email, String cpf)
    {
        super(id, nome, logradouro, cidade, estado, telefone, email);
        this.cpf = cpf;
    }

    public String getCpf()
    {
        return cpf;
    }

    @Override
    public void exibir()
    {
        super.exibir();
        System.out.println("CPF: " + cpf);
    }

    @Override
    public void setID(int newID)
    {
        super.setID(newID);
    }

    @Override
    public int getId()
    {
        return super.getId();
    }
}
```

```
package cadastrobd.model;

public class PessoaJuridica extends Pessoa
{
    private String cnpj;

    public PessoaJuridica()
    {
        super();
    }

    public PessoaJuridica(String nome, String logradouro, String cidade, String estado,
String telefone, String email, String cnpj)
```



```
    {
        super(nome, logradouro, cidade, estado, telefone, email);
        this.cnpj = cnpj;
    }

    public PessoaJuridica(int id, String nome, String logradouro, String cidade, String
estado, String telefone, String email, String cnpj)
    {
        super(id, nome, logradouro, cidade, estado, telefone, email);
        this.cnpj = cnpj;
    }

    public String getCnpj()
    {
        return cnpj;
    }

    @Override
    public void exibir()
    {
        super.exibir();
        System.out.println("CNPJ: " + cnpj);
    }

    @Override
    public void setID(int newID)
    {
        super.setID(newID);
    }

    @Override
    public int getId()
    {
        return super.getId();
    }
}
```

```
package cadastro.model.util;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class ConectorBD
{
    private static final String URL = "jdbc:sqlserver://localhost\
\SQLEXPRESS:1433;databaseName=loja;encrypt=true;trustServerCertificate=true;";
    private static final String USUARIO = "loja";
    private static final String SENHA = "loja";
```

```
public static Connection getConnection()
{
    try
    {
        return DriverManager.getConnection(URL, USUARIO, SENHA);
    }
    catch (SQLException e)
    {
        System.err.println("Erro ao conectar: " + e.getMessage());
        return null;
    }
}

public static PreparedStatement getPrepared(String sql)
{
    try
    {
        Connection conn = getConnection();
        if (conn != null)
        {
            return conn.prepareStatement(sql);
        }
        else
        {
            throw new SQLException("Conexão nula. Verifique o banco de dados.");
        }
    }
    catch (SQLException e)
    {
        System.err.println("Erro ao preparar statement: " + e.getMessage());
        return null;
    }
}

public static ResultSet getSelect(String sql)
{
    try
    {
        PreparedStatement ps = getPrepared(sql);
        if (ps != null)
        {
            return ps.executeQuery();
        }
        else
        {
            throw new SQLException("PreparedStatement nulo. Verifique o SQL.");
        }
    }
    catch (SQLException e)
    {
```

```
            System.err.println("Erro ao executar select: " + e.getMessage());
            return null;
        }
    }

    public static void close(ResultSet rs)
    {
        if (rs != null)
        {
            try
            {
                rs.close();
            }
            catch (SQLException e)
            {
                System.err.println("Erro ao fechar ResultSet: " + e.getMessage());
            }
        }
    }

    public static void close(Statement st)
    {
        if (st != null)
        {
            try
            {
                st.close();
            }
            catch (SQLException e)
            {
                System.err.println("Erro ao fechar Statement: " + e.getMessage());
            }
        }
    }

    public static void close(Connection conn)
    {
        if (conn != null)
        {
            try
            {
                conn.close();
            }
            catch (SQLException e)
            {
                System.err.println("Erro ao fechar Connection: " + e.getMessage());
            }
        }
    }
}
```

```
package cadastro.model.util;
```

```
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

public class SequenceManager
{
   public static int getValue(String sequenceName)
   {
      int valor = -1;

      String sql = "UPDATE sequences SET valor = valor + 1 WHERE nome = ?";
      String selectSql = "SELECT valor FROM sequences WHERE nome = ?";

      Connection conn = null;
      PreparedStatement updateStmt = null;
      PreparedStatement selectStmt = null;
      ResultSet rs = null;

      try
      {
         conn = ConectorBD.getConnection();

         updateStmt = conn.prepareStatement(sql);
         updateStmt.setString(1, sequenceName);
         int affected = updateStmt.executeUpdate();

         if (affected == 0)
         {
            throw new SQLException("Sequência '" + sequenceName + "' não
encontrada.");
         }

         selectStmt = conn.prepareStatement(selectSql);
         selectStmt.setString(1, sequenceName);
         rs = selectStmt.executeQuery();

         if (rs.next())
         {
            valor = rs.getInt("valor");
         }

      }
      catch (SQLException e)
      {
         System.err.println("Erro ao obter valor da sequência: " + e.getMessage());
      }
      finally
      {
         ConectorBD.close(rs);
```

```
            ConectorBD.close(updateStmt);
            ConectorBD.close(selectStmt);
            ConectorBD.close(conn);
        }

        return valor;
    }
}
```

```
package cadastrobd.model;

import cadastro.model.util.ConectorBD;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class PessoaFisicaDAO
{
    public PessoaFisica getPessoa(int id)
    {
        PessoaFisica pf = null;
        Connection conn = null;
        PreparedStatement ps = null;
        ResultSet rs = null;

        String sql = """
            SELECT p.idPessoa , p.nome, p.logradouro, p.cidade, p.estado, p.telefone,
p.email, pf.cpf
            FROM Pessoa p
            JOIN PessoaFisica pf ON p.idPessoa  = pf.idPessoa
            WHERE p.idPessoa  = ?
        """;

        try
        {
            conn = ConectorBD.getConnection();
            ps = conn.prepareStatement(sql);
            ps.setInt(1, id);
            rs = ps.executeQuery();

            if (rs.next())
            {
                pf = new PessoaFisica(
                    rs.getInt("idPessoa"),
                    rs.getString("nome"),
                    rs.getString("logradouro"),
                    rs.getString("cidade"),
                    rs.getString("estado"),
                    rs.getString("telefone"),
                    rs.getString("email"),
                    rs.getString("cpf")
                );
```

```
            }
        }
        catch (SQLException e)
        {
            System.err.println("Erro ao buscar Pessoa Física: " + e.getMessage());
        }
        finally
        {
            ConectorBD.close(rs);
            ConectorBD.close(ps);
            ConectorBD.close(conn);
        }

        return pf;
    }

    public List<PessoaFisica> getPessoas()
    {
        List<PessoaFisica> lista = new ArrayList<>();
        Connection conn = null;
        PreparedStatement ps = null;
        ResultSet rs = null;

        String sql = """
            SELECT p.idPessoa , p.nome, p.logradouro, p.cidade, p.estado, p.telefone,
p.email, pf.cpf
            FROM Pessoa p
            JOIN PessoaFisica pf ON p.idPessoa  = pf.idPessoa
        """;

        try
        {
            conn = ConectorBD.getConnection();
            ps = conn.prepareStatement(sql);
            rs = ps.executeQuery();

            while (rs.next())
            {
                PessoaFisica pf = new PessoaFisica(
                    rs.getInt("idPessoa"),
                    rs.getString("nome"),
                    rs.getString("logradouro"),
                    rs.getString("cidade"),
                    rs.getString("estado"),
                    rs.getString("telefone"),
                    rs.getString("email"),
                    rs.getString("cpf")
                );
                lista.add(pf);
            }
        }
```

```
        catch (SQLException e)
        {
            System.err.println("Erro ao listar Pessoas Físicas: " + e.getMessage());
        }
        finally
        {
            ConectorBD.close(rs);
            ConectorBD.close(ps);
            ConectorBD.close(conn);
        }

        return lista;
    }

    public boolean incluir(PessoaFisica pf)
    {
        Connection conn = null;
        PreparedStatement psPessoa = null;
        PreparedStatement psFisica = null;
        ResultSet rs = null;

        String sqlPessoa = """
            INSERT INTO Pessoa (nome, logradouro, cidade, estado, telefone, email)
            VALUES (?, ?, ?, ?, ?, ?)
        """;

        String sqlFisica = """
            INSERT INTO PessoaFisica (idPessoa, cpf)
            VALUES (?, ?)
        """;

        try {
            conn = ConectorBD.getConnection();

            psPessoa = conn.prepareStatement(sqlPessoa,
Statement.RETURN_GENERATED_KEYS);
            psPessoa.setString(1, pf.getNome());
            psPessoa.setString(2, pf.getLogradouro());
            psPessoa.setString(3, pf.getCidade());
            psPessoa.setString(4, pf.getEstado());
            psPessoa.setString(5, pf.getTelefone());
            psPessoa.setString(6, pf.getEmail());
            psPessoa.executeUpdate();

            rs = psPessoa.getGeneratedKeys();
            if (rs.next()) {
                int idGerado = rs.getInt(1);
                pf.setID(idGerado);

                psFisica = conn.prepareStatement(sqlFisica);
                psFisica.setInt(1, idGerado);
```

```
            psFisica.setString(2, pf.getCpf());
            psFisica.executeUpdate();

            return true;
        } else {
            System.err.println("Erro ao obter o ID gerado.");
            return false;
        }
    } catch (SQLException e) {
        System.err.println("Erro ao incluir Pessoa Física: " + e.getMessage());
        return false;
    } finally {
        ConectorBD.close(rs);
        ConectorBD.close(psPessoa);
        ConectorBD.close(psFisica);
        ConectorBD.close(conn);
    }
}

public boolean alterar(PessoaFisica pf)
{
    Connection conn = null;
    PreparedStatement psPessoa = null;
    PreparedStatement psFisica = null;

    String sqlPessoa = """
        UPDATE Pessoa
        SET nome = ?, logradouro = ?, cidade = ?, estado = ?, telefone = ?, email = ?
        WHERE idPessoa  = ?
    """;

    String sqlFisica = """
        UPDATE PessoaFisica
        SET cpf = ?
        WHERE idPessoa  = ?
    """;

    try
    {
        conn = ConectorBD.getConnection();

        psPessoa = conn.prepareStatement(sqlPessoa);
        psPessoa.setString(1, pf.getNome());
        psPessoa.setString(2, pf.getLogradouro());
        psPessoa.setString(3, pf.getCidade());
        psPessoa.setString(4, pf.getEstado());
        psPessoa.setString(5, pf.getTelefone());
        psPessoa.setString(6, pf.getEmail());
        psPessoa.setInt(7, pf.getId());
        psPessoa.executeUpdate();
```

```
            psFisica = conn.prepareStatement(sqlFisica);
            psFisica.setString(1, pf.getCpf());
            psFisica.setInt(2, pf.getId());
            psFisica.executeUpdate();

            return true;
        }
        catch (SQLException e)
        {
            System.err.println("Erro ao alterar Pessoa Física: " + e.getMessage());
            return false;
        }
        finally
        {
            ConectorBD.close(psPessoa);
            ConectorBD.close(psFisica);
            ConectorBD.close(conn);
        }
    }

    public boolean excluir(int id)
    {
        Connection conn = null;
        PreparedStatement psFisica = null;
        PreparedStatement psPessoa = null;

        String sqlFisica = "DELETE FROM PessoaFisica WHERE idPessoa  = ?";
        String sqlPessoa = "DELETE FROM Pessoa WHERE idPessoa  = ?";

        try
        {
            conn = ConectorBD.getConnection();

            psFisica = conn.prepareStatement(sqlFisica);
            psFisica.setInt(1, id);
            psFisica.executeUpdate();

            psPessoa = conn.prepareStatement(sqlPessoa);
            psPessoa.setInt(1, id);
            psPessoa.executeUpdate();

            return true;
        }
        catch (SQLException e)
        {
            System.err.println("Erro ao excluir Pessoa Física: " + e.getMessage());
            return false;
        }
        finally
        {
            ConectorBD.close(psFisica);
```



```
            ConectorBD.close(psPessoa);
            ConectorBD.close(conn);
        }
    }
}
```

```
package cadastrobd.model;

import cadastro.model.util.ConectorBD;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class PessoaJuridicaDAO
{
    public PessoaJuridica getPessoa(int id)
    {
        PessoaJuridica pj = null;
        Connection conn = null;
        PreparedStatement ps = null;
        ResultSet rs = null;

        String sql = """
            SELECT p.idPessoa , p.nome, p.logradouro, p.cidade, p.estado, p.telefone,
p.email, pf.cnpj
            FROM Pessoa p
            JOIN PessoaJuridica pf ON p.idPessoa  = pf.idPessoa
            WHERE p.idPessoa  = ?
        """;

        try
        {
            conn = ConectorBD.getConnection();
            ps = conn.prepareStatement(sql);
            ps.setInt(1, id);
            rs = ps.executeQuery();

            if (rs.next())
            {
                pj = new PessoaJuridica(
                    rs.getInt("idPessoa"),
                    rs.getString("nome"),
                    rs.getString("logradouro"),
                    rs.getString("cidade"),
                    rs.getString("estado"),
                    rs.getString("telefone"),
                    rs.getString("email"),
                    rs.getString("cnpj")
                );
            }
        }
        catch (SQLException e)
```

```
        {
            System.err.println("Erro ao buscar Pessoa Física: " + e.getMessage());
        }
        finally
        {
            ConectorBD.close(rs);
            ConectorBD.close(ps);
            ConectorBD.close(conn);
        }

        return pj;
    }

    public List<PessoaJuridica> getPessoas()
    {
        List<PessoaJuridica> lista = new ArrayList<>();
        Connection conn = null;
        PreparedStatement ps = null;
        ResultSet rs = null;

        String sql = """
            SELECT p.idPessoa , p.nome, p.logradouro, p.cidade, p.estado, p.telefone,
p.email, pf.cnpj
            FROM Pessoa p
            JOIN PessoaJuridica pf ON p.idPessoa  = pf.idPessoa
        """;

        try
        {
            conn = ConectorBD.getConnection();
            ps = conn.prepareStatement(sql);
            rs = ps.executeQuery();

            while (rs.next())
            {
                PessoaJuridica pf = new PessoaJuridica(
                    rs.getInt("idPessoa"),
                    rs.getString("nome"),
                    rs.getString("logradouro"),
                    rs.getString("cidade"),
                    rs.getString("estado"),
                    rs.getString("telefone"),
                    rs.getString("email"),
                    rs.getString("cnpj")
                );
                lista.add(pf);
            }
        }
        catch (SQLException e)
        {
            System.err.println("Erro ao listar Pessoa Juridica: " + e.getMessage());
```

```
        }
        finally
        {
            ConectorBD.close(rs);
            ConectorBD.close(ps);
            ConectorBD.close(conn);
        }

        return lista;
    }

    public boolean incluir(PessoaJuridica pj)
    {
        Connection conn = null;
        PreparedStatement psPessoa = null;
        PreparedStatement psJuridica = null;
        ResultSet rs = null;

        String sqlPessoa = """
            INSERT INTO Pessoa (nome, logradouro, cidade, estado, telefone, email)
            VALUES (?, ?, ?, ?, ?, ?)
        """;

        String sqlJuridica = """
            INSERT INTO PessoaJuridica (idPessoa, cnpj)
            VALUES (?, ?)
        """;

        try {
            conn = ConectorBD.getConnection();

            psPessoa = conn.prepareStatement(sqlPessoa,
Statement.RETURN_GENERATED_KEYS);
            psPessoa.setString(1, pj.getNome());
            psPessoa.setString(2, pj.getLogradouro());
            psPessoa.setString(3, pj.getCidade());
            psPessoa.setString(4, pj.getEstado());
            psPessoa.setString(5, pj.getTelefone());
            psPessoa.setString(6, pj.getEmail());
            psPessoa.executeUpdate();

            rs = psPessoa.getGeneratedKeys();
            if (rs.next()) {
                int idGerado = rs.getInt(1);
                pj.setID(idGerado);

                psJuridica = conn.prepareStatement(sqlJuridica);
                psJuridica.setInt(1, idGerado);
                psJuridica.setString(2, pj.getCnpj());
                psJuridica.executeUpdate();
```

```
            return true;
        } else {
            System.err.println("Erro ao obter o ID gerado.");
            return false;
        }
    } catch (SQLException e) {
        System.err.println("Erro ao incluir Pessoa Juridica: " + e.getMessage());
        return false;
    } finally {
        ConectorBD.close(rs);
        ConectorBD.close(psPessoa);
        ConectorBD.close(psJuridica);
        ConectorBD.close(conn);
    }
}

public boolean alterar(PessoaJuridica pj)
{
    Connection conn = null;
    PreparedStatement psPessoa = null;
    PreparedStatement psJuridica = null;

    String sqlPessoa = """
        UPDATE Pessoa
        SET nome = ?, logradouro = ?, cidade = ?, estado = ?, telefone = ?, email = ?
        WHERE idPessoa  = ?
    """;

    String sqlJuridica = """
        UPDATE PessoaJuridica
        SET cnpj = ?
        WHERE idPessoa  = ?
    """;

    try
    {
        conn = ConectorBD.getConnection();

        psPessoa = conn.prepareStatement(sqlPessoa);
        psPessoa.setString(1, pj.getNome());
        psPessoa.setString(2, pj.getLogradouro());
        psPessoa.setString(3, pj.getCidade());
        psPessoa.setString(4, pj.getEstado());
        psPessoa.setString(5, pj.getTelefone());
        psPessoa.setString(6, pj.getEmail());
        psPessoa.setInt(7, pj.getId());
        psPessoa.executeUpdate();

        psJuridica = conn.prepareStatement(sqlJuridica);
        psJuridica.setString(1, pj.getCnpj());
        psJuridica.setInt(2, pj.getId());
```

```
            psJuridica.executeUpdate();

            return true;
        }
        catch (SQLException e)
        {
            System.err.println("Erro ao alterar Pessoa Juridica: " + e.getMessage());
            return false;
        }
        finally
        {
            ConectorBD.close(psPessoa);
            ConectorBD.close(psJuridica);
            ConectorBD.close(conn);
        }
    }

    public boolean excluir(int id)
    {
        Connection conn = null;
        PreparedStatement psJuridica = null;
        PreparedStatement psPessoa = null;

        String sqlJuridica = "DELETE FROM PessoaJuridica WHERE idPessoa  = ?";
        String sqlPessoa = "DELETE FROM Pessoa WHERE idPessoa  = ?";

        try
        {
            conn = ConectorBD.getConnection();

            psJuridica = conn.prepareStatement(sqlJuridica);
            psJuridica.setInt(1, id);
            psJuridica.executeUpdate();

            psPessoa = conn.prepareStatement(sqlPessoa);
            psPessoa.setInt(1, id);
            psPessoa.executeUpdate();

            return true;
        }
        catch (SQLException e)
        {
            System.err.println("Erro ao excluir Pessoa Juridica: " + e.getMessage());
            return false;
        }
        finally
        {
            ConectorBD.close(psJuridica);
            ConectorBD.close(psPessoa);
            ConectorBD.close(conn);
        }
```

```
    }
}
```

```
package cadastrobd;

import cadastrobd.model.PessoaFisica;
import cadastrobd.model.PessoaJuridica;
import cadastrobd.model.PessoaFisicaDAO;
import cadastrobd.model.PessoaJuridicaDAO;
import java.util.Collections;
import java.util.List;
import java.util.Scanner;

public class CadastroBDTeste
{
    public static void main(String[] args)
    {
        Scanner scanner = new Scanner(System.in);
        PessoaFisicaDAO pfDao = new PessoaFisicaDAO();
        PessoaJuridicaDAO pjDao = new PessoaJuridicaDAO();
        try
        {
            String nome = "João";
            String logradouro = "Rua Qualquer";
            String cidade = "Cidade";
            String estado = "Estado";
            String telefone = "999999999";
            String email = "email@exemplo.com";
            String cpf = "999999999";

            PessoaFisica Pf = criarPessoaFisica(scanner, nome, logradouro, cidade,
estado, telefone, email, cpf);
            pfDao.incluir(Pf);

            nome = "João Lucas";
            logradouro = "Rua Qualquer";
            cidade = "Cidade";
            estado = "Estado";
            telefone = "999999999";
            email = "email@exemplo.com";
            cpf = "999999999";

            Pf = criarPessoaFisica(scanner, nome, logradouro, cidade, estado, telefone,
email, cpf, Pf.getId());
            pfDao.alterar(Pf);

            exibirCabecalho("F");
            for (PessoaFisica pf : pfDao.getPessoas())
            {
                System.out.printf("%-5d %-20s %-20s %-15s %-10s %-15s %-25s
%-14s%n",
                    pf.getId(),
```

```
                pf.getNome(),
                pf.getLogradouro(),
                pf.getCidade(),
                pf.getEstado(),
                pf.getTelefone(),
                pf.getEmail(),
                pf.getCpf());
        }

        pfDao.excluir(Pf.getId());

        nome = "Empresa X";
        logradouro = "Rua Qualquer";
        cidade = "Cidade";
        estado = "Estado";
        telefone = "999999999";
        email = "emailempresa@exemplo.com";
        String cnpj = "999999999";

        PessoaJuridica Pj = criarPessoaJuridica(scanner, nome, logradouro, cidade,
estado, telefone, email, cnpj);
        pjDao.incluir(Pj);

        nome = "Empresa Y";
        logradouro = "Rua Qualquer";
        cidade = "Cidade";
        estado = "Estado";
        telefone = "999999999";
        email = "emailempresa@exemplo.com";
        cnpj = "999999999";

        Pj = criarPessoaJuridica(scanner, nome, logradouro, cidade, estado, telefone,
email, cnpj, Pj.getId());
        pjDao.alterar(Pj);

        exibirCabecalho("J");
        for (PessoaJuridica pj : pjDao.getPessoas())
        {
           System.out.printf("%-5d %-20s %-20s %-15s %-10s %-15s %-25s
%-14s%n",
                pj.getId(),
                pj.getNome(),
                pj.getLogradouro(),
                pj.getCidade(),
                pj.getEstado(),
                pj.getTelefone(),
                pj.getEmail(),
                pj.getCnpj());
        }

        pjDao.excluir(Pj.getId());
```

```
        }
        catch (Exception e)
        {
            System.out.println("Erro: " + e.getMessage());
            e.printStackTrace();
        }
    }

    public static void exibirCabecalho(String tipo)
    {
        System.out.printf("%-5s %-20s %-20s %-15s %-10s %-15s %-25s %-14s%n",
            "ID", "Nome", "Logradouro", "Cidade", "Estado", "Telefone", "Email",
tipo.equals("F") ? "CPF" : "CNPJ");
        System.out.println("=".repeat(130));
    }

    public static void exibirPessoasFisicas(List<PessoaFisica> lista)
    {
        for (PessoaFisica pf : lista) {
            System.out.printf("%-5d %-20s %-20s %-15s %-10s %-15s %-25s %-14s%n",
                pf.getId(),
                pf.getNome(),
                pf.getLogradouro(),
                pf.getCidade(),
                pf.getEstado(),
                pf.getTelefone(),
                pf.getEmail(),
                pf.getCpf());
        }
    }

    public static void exibirPessoasJuridicas(List<PessoaJuridica> lista)
    {
        for (PessoaJuridica pj : lista) {
            System.out.printf("%-5d %-20s %-20s %-15s %-10s %-15s %-25s %-14s%n",
                pj.getId(),
                pj.getNome(),
                pj.getLogradouro(),
                pj.getCidade(),
                pj.getEstado(),
                pj.getTelefone(),
                pj.getEmail(),
                pj.getCnpj());
        }
    }

    public static PessoaJuridica criarPessoaJuridica(Scanner scanner, String nome,
String logradouro, String cidade, String estado, String telefone, String email, String
cnpj)
    {
        return new PessoaJuridica(nome, logradouro, cidade, estado, telefone, email,
```

```
cnpj);
    }

    public static PessoaFisica criarPessoaFisica(Scanner scanner, String nome, String
logradouro, String cidade, String estado, String telefone, String email, String cpf)
    {
        return new PessoaFisica(nome, logradouro, cidade, estado, telefone, email, cpf);
    }

    public static PessoaJuridica criarPessoaJuridica(Scanner scanner, String nome,
String logradouro, String cidade, String estado, String telefone, String email, String
cnpj, int id)
    {
        return new PessoaJuridica(id, nome, logradouro, cidade, estado, telefone, email,
cnpj);
    }

    public static PessoaFisica criarPessoaFisica(Scanner scanner, String nome, String
logradouro, String cidade, String estado, String telefone, String email, String cpf, int
id)
    {
        return new PessoaFisica(id, nome, logradouro, cidade, estado, telefone, email,
cpf);
    }
}
```

a) Qual a importância dos componentes de middleware, como o JDBC?

R: É importante para poder conectar aplicativos com bancos de dados em suas utilidades.

b) Qual a diferença no uso de ou para a manipulação de dados?

R: PreparedStatement é mais seguro e rápido quando se trata de comandos repetidos enquanto que Statement é melhor para comandos mais simples.

c) Como o padrão DAO melhora a manutenibilidade do software?

R: Para organizar o projeto, o padrão DAO separa a lógica de acesso a dados do resto do código.

d) Como a herança é refletida no banco de dados, quando lidamos com um modelo estritamente relacional?

R: Pode se usar uma tabela única com uma coluna descriminatória, uma tabela por classe concreta ou tabelas separadas mas com junção

2º Procedimento | Alimentando a Base

```
package cadastrobd;

import cadastrobd.model.PessoaFisica;
import cadastrobd.model.PessoaJuridica;
import cadastrobd.model.PessoaFisicaDAO;
import cadastrobd.model.PessoaJuridicaDAO;
import java.util.Collections;
import java.util.List;
import java.util.Scanner;

public class CadastroBD {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        PessoaFisicaDAO pfDao = new PessoaFisicaDAO();
        PessoaJuridicaDAO pjDao = new PessoaJuridicaDAO();
        int opcao = 0;

        do
        {
            try
            {
                System.out.println("\n===== MENU =====");
                System.out.println("1 - Incluir");
                System.out.println("2 - Alterar");
                System.out.println("3 - Excluir");
                System.out.println("4 - Exibir por ID");
                System.out.println("5 - Exibir todos");
                System.out.println("0 - Sair");
                System.out.print("Escolha uma opção: ");
                opcao = Integer.parseInt(scanner.nextLine());

                switch (opcao)
                {
                    case 1:
                    {
                        System.out.print("Tipo (F para Física / J para Jurídica): ");
                        String tipo = scanner.nextLine().toUpperCase();

                        if (tipo.equals("F"))
                        {
                            PessoaFisica pf = inputPessoasFisicas(scanner);
                            pfDao.incluir(pf);

                            exibirCabecalho("F");
                            exibirPessoasFisicas(Collections.singletonList(pf));
                        }
                        else if (tipo.equals("J"))
                        {
                            PessoaJuridica pj = inputPessoasJuridicas(scanner);
                            pjDao.incluir(pj);
```

```
                exibirCabecalho("J");
                exibirPessoasJuridicas(Collections.singletonList(pj));
            }
            break;
        }
        case 2:
        {
            System.out.print("Tipo (F para Física / J para Jurídica): ");
            String tipo = scanner.nextLine().toUpperCase();

            if (tipo.equals("F"))
            {
                System.out.print("ID da pessoa a ser alterada: ");
                int targetId = Integer.parseInt(scanner.nextLine());

                PessoaFisica pessoaCheckF = pfDao.getPessoa(targetId);
                if (pessoaCheckF != null)
                {
                    exibirCabecalho("F");
                    exibirPessoasFisicas(Collections.singletonList(pessoaCheckF));

                    PessoaFisica pf = inputPessoasFisicas(scanner);
                    pf.setID(targetId);
                    pfDao.alterar(pf);

                    exibirCabecalho("F");
                    exibirPessoasFisicas(Collections.singletonList(pf));
                }
                else
                {
                    System.out.println("Pessoa Física não encontrada.");
                }

            }
            else if (tipo.equals("J"))
            {
                System.out.print("ID da pessoa a ser alterada: ");
                int targetId = Integer.parseInt(scanner.nextLine());

                PessoaJuridica pessoaCheckJ = pjDao.getPessoa(targetId);
                if (pessoaCheckJ != null)
                {
                    exibirCabecalho("J");
                    exibirPessoasJuridicas(Collections.singletonList(pessoaCheckJ));

                    PessoaJuridica pj = inputPessoasJuridicas(scanner);
                    pj.setID(targetId);
                    pjDao.alterar(pj);

                    exibirCabecalho("J");
```

```
                exibirPessoasJuridicas(Collections.singletonList(pj));
            }
            else
            {
                System.out.println("Pessoa Jurídica não encontrada.");
            }
        }
        break;
    }

    case 3:
    {
        System.out.print("Tipo (F para Física / J para Jurídica): ");
        String tipo = scanner.nextLine().toUpperCase();
        if (tipo.equals("F"))
        {
            System.out.print("ID: ");
            int idExc = Integer.parseInt(scanner.nextLine());
            PessoaFisica pessoaCheckF = pfDao.getPessoa(idExc);

            if (pessoaCheckF != null)
            {
                exibirCabecalho("F");
                exibirPessoasFisicas(Collections.singletonList(pessoaCheckF));

                System.out.printf("Tem certeza que deseja excluir? s/n");
                String confirm = scanner.nextLine();
                if ("s".equals(confirm) || "S".equals(confirm))
                {
                    pfDao.excluir(idExc);
                    System.out.print("Pessoa excluida com successo");
                }
                else
                {
                    System.out.print("Cancelado");
                }
            }
            else
            {
                System.out.print("Pessoa Fisica não encontrada.");
            }
        }
        else if (tipo.equals("J"))
        {
            System.out.print("ID: ");
            int idExc = Integer.parseInt(scanner.nextLine());
            PessoaJuridica pessoaCheckJ = pjDao.getPessoa(idExc);

            if (pessoaCheckJ != null)
            {
                exibirCabecalho("J");
```

```
                    exibirPessoasJuridicas(Collections.singletonList(pessoaCheckJ));

                    System.out.printf("Tem certeza que deseja excluir? s/n");
                    String confirm = scanner.nextLine();
                    if ("s".equals(confirm) || "S".equals(confirm))
                    {
                        pjDao.excluir(idExc);
                        System.out.print("Pessoa excluida com successo");
                    }
                    else
                    {
                        System.out.print("Cancelado");
                    }
                }
                else
                {
                    System.out.print("Pessoa Jurídica não encontrada.");
                }
            }
            break;
        }

        case 4:
        {
            System.out.print("Tipo (F para Física / J para Jurídica): ");
            String tipo = scanner.nextLine().toUpperCase();
            System.out.print("ID: ");
            int idExibir = Integer.parseInt(scanner.nextLine());
            if (tipo.equals("F"))
            {
                PessoaFisica pf = pfDao.getPessoa(idExibir);
                if (pf != null)
                {
                    exibirCabecalho("F");
                    exibirPessoasFisicas(Collections.singletonList(pf));
                }
                else
                {
                    System.out.println("Pessoa Física não encontrada.");
                }
            }
            else if (tipo.equals("J"))
            {
                PessoaJuridica pj = pjDao.getPessoa(idExibir);
                if (pj != null)
                {
                    exibirCabecalho("J");
                    exibirPessoasJuridicas(Collections.singletonList(pj));
                }
                else
                {
```

```
                        System.out.println("Pessoa Jurídica não encontrada.");
                    }
                }
                break;
            }

            case 5:
            {
                System.out.print("Tipo (F para Física / J para Jurídica): ");
                String tipo = scanner.nextLine().toUpperCase();
                if (tipo.equals("F"))
                {
                    exibirCabecalho("F");
                    exibirPessoasFisicas(pfDao.getPessoas());
                }
                else if (tipo.equals("J"))
                {
                    exibirCabecalho("J");
                    exibirPessoasJuridicas(pjDao.getPessoas());
                }
                break;
            }

            case 0:
            {
                System.out.println("Encerrando o sistema...");
                break;
            }

            default:
            {
                System.out.println("Opção inválida.");
                break;
            }
        }
    }
    catch (Exception e)
    {
        System.out.println("Erro: " + e.getMessage());
        e.printStackTrace();
    }
}
while (opcao != 0);

scanner.close();
}

public static void exibirCabecalho(String tipo)
{
    System.out.printf("%-5s %-20s %-20s %-15s %-10s %-15s %-25s %-14s%n",
        "ID", "Nome", "Logradouro", "Cidade", "Estado", "Telefone", "Email",
```

```
tipo.equals("F") ? "CPF" : "CNPJ");
        System.out.println("=".repeat(130));
    }

    public static void exibirPessoasFisicas(List<PessoaFisica> lista)
    {
        for (PessoaFisica pf : lista) {
            System.out.printf("%-5d %-20s %-20s %-15s %-10s %-15s %-25s %-14s%n",
                pf.getId(),
                pf.getNome(),
                pf.getLogradouro(),
                pf.getCidade(),
                pf.getEstado(),
                pf.getTelefone(),
                pf.getEmail(),
                pf.getCpf());
        }
    }

    public static void exibirPessoasJuridicas(List<PessoaJuridica> lista)
    {
        for (PessoaJuridica pj : lista) {
            System.out.printf("%-5d %-20s %-20s %-15s %-10s %-15s %-25s %-14s%n",
                pj.getId(),
                pj.getNome(),
                pj.getLogradouro(),
                pj.getCidade(),
                pj.getEstado(),
                pj.getTelefone(),
                pj.getEmail(),
                pj.getCnpj());
        }
    }

    public static PessoaFisica inputPessoasFisicas(Scanner scanner)
    {
        System.out.print("Nome: ");
        String nome = scanner.nextLine();
        System.out.print("Logradouro: ");
        String logradouro = scanner.nextLine();
        System.out.print("Cidade: ");
        String cidade = scanner.nextLine();
        System.out.print("Estado: ");
        String estado = scanner.nextLine();
        System.out.print("Telefone: ");
        String telefone = scanner.nextLine();
        System.out.print("Email: ");
        String email = scanner.nextLine();
        System.out.print("CPF: ");
        String cpf = scanner.nextLine();
```

```
        return new PessoaFisica(nome, logradouro, cidade, estado, telefone, email, cpf);
    }

    public static PessoaJuridica inputPessoasJuridicas(Scanner scanner)
    {
        System.out.print("Nome: ");
        String nome = scanner.nextLine();
        System.out.print("Logradouro: ");
        String logradouro = scanner.nextLine();
        System.out.print("Cidade: ");
        String cidade = scanner.nextLine();
        System.out.print("Estado: ");
        String estado = scanner.nextLine();
        System.out.print("Telefone: ");
        String telefone = scanner.nextLine();
        System.out.print("Email: ");
        String email = scanner.nextLine();
        System.out.print("CNPJ: ");
        String cnpj = scanner.nextLine();

        return new PessoaJuridica(nome, logradouro, cidade, estado, telefone, email,
cnpj);
    }
}
```

a) Quais as diferenças entre a persistência em arquivo e a persistência em banco de dados?

R: Persistência em arquivo é mais simples mas bancos de dados são mais escalonáveis e são ideais para projetos mais complexos.

b) Como o uso de operador simplificou a impressão dos valores contidos nas entidades, nas versões mais recentes do Java?

R: O lambda no Java permite escrever código mais resumido e direto.

c) Por que métodos acionados diretamente pelo método main, sem o uso de um objeto, precisam ser marcados como ?

R: Porque o método main é static, logo, ele roda sem criar um objeto da classe. Então ele só pode chamar diretamente métodos que também sejam static.